



МИНОБРНАУКИ РОССИИ

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра прикладной математики (ПМ)**

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №4
по дисциплине «Технологии и инструментарий машинного обучения»

Студент группы *ИНБО-20-23. Боргачев Т.М.*

(подпись)

Преподаватель *Трушин СМ*

(подпись)

Москва 2025 г.

СОДЕРЖАНИЕ

1 ПРАКТИЧЕСКАЯ РАБОТА №4	3
1.1 Задание 1. Линейные модели	3
1.1.1 Выбор датасета	3
1.1.2 Предобработка и первичный анализ данных	4
1.1.3 Построение модели логистической регрессии	7
1.1.4 Оценка моделей	9
1.1.5 Сравнение результатов и сред.....	11
1.2 Задание 2. Классификация при несбалансированности классов	11
1.2.1 Выбор датасета	11
1.2.2 Проверка распределения классов	12
1.2.3 Логистическая регрессия без балансировки классов	13
1.2.4 Выбор метода устранения дисбаланса	15
1.2.5 Повторное обучение	15
1.3 Задание 3. Многоклассовая логистическая регрессия.....	17
1.3.1 Выбор датасета	17
1.3.2 Постройка моделей.....	18
ВЫВОД.....	22

1 ПРАКТИЧЕСКАЯ РАБОТА №4

1.1 Задание 1. Линейные модели

1.1.1 Выбор датасета

Ссылка на Google colab с выполненной работой:
https://colab.research.google.com/drive/1nr0qn9ZLuJBWmmfeB1okir_zOJZsyo4u?usp=sharing.

Выбранный датасет: «Adult.data» (Прогнозирование того, превышает ли годовой доход физического лица 50 тысяч долларов в год, на основе данных переписи. Также известный как набор данных "Доходы от переписи".) — в нем есть множество числовых и категориальных признаков, пропуски в данных, больше 1000 строк (48842), а также он может быть использован для классификации: доход $>50K$ или $\leq 50K$.

Код для обзора данных представлен в Листинге 1.

Листинг 1 — Обзор данных в Python

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from ucimlrepo import fetch_ucirepo

adult = fetch_ucirepo(id=2)

X = adult.data.features
y = adult.data.targets
df = pd.concat([X, y], axis=1)
df.head()
print(df.shape)
print("\nПропуски:")
print(df.isnull().sum())
print('Дубликаты:', df.duplicated().sum())
print("\nОписательная статистика:")
print(df.describe(include='all'))
```

Основные данные, пропуски и дубликаты представлены на Рисунке 1.

	age	workclass	fnlwgt	education	education-num	marital-status	occupation	relationship	race	sex	capital-gain	capital-loss	hours-per-week	native-country	income
0	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White	Male	2174	0	40	United-States	<=50K
1	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	13	United-States	<=50K
2	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male	0	0	40	United-States	<=50K
3	53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	0	0	40	United-States	<=50K
4	28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife	Black	Female	0	0	40	Cuba	<=50K


```
✓ print(df.shape) ...
```

(48842, 15)

Пропуски:

age	0
workclass	963
fnlwgt	0
education	0
education-num	0
marital-status	0
occupation	966
relationship	0
race	0
sex	0
capital-gain	0
capital-loss	0
hours-per-week	0
native-country	274
income	0

dtype: int64
Дубликаты: 29

Рисунок 1 — Обзор данных

Загрузка данных в Logiном представлена на Рисунке 2.

Logiном Community

Сценарий

Adult.csv • Быстрый просмотр

Набор данных

#	12 age	ab workclass	12 fnlwgt	ab education	12 education-num	ab marital-status	ab occupation	ab relationship	ab race	ab sex	12 capital-gain	12 capital-loss	12 hours-per-week
1	39	State-gov	77 516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White	Male	2 174	0	
2	50	Self-emp-not-inc	83 311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	
3	38	Private	215 646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male	0	0	
4	53	Private	234 721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	0	0	
5	28	Private	338 409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife	Black	Female	0	0	
6	37	Private	284 582	Masters	14	Married-civ-spouse	Exec-managerial	Wife	White	Female	0	0	
7	49	Private	160 187	9th	5	Married-spouse-absent	Other-service	Not-in-family	Black	Female	0	0	
8	52	Self-emp-not-inc	209 642	HS-grad	9	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	
9	31	Private	45 781	Masters	14	Never-married	Prof-specialty	Not-in-family	White	Female	14 084	0	
10	42	Private	159 449	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	5 178	0	
11	37	Private	280 464	Some-college	10	Married-civ-spouse	Exec-managerial	Husband	Black	Male	0	0	
12	30	State-gov	141 297	Bachelors	13	Married-civ-spouse	Prof-specialty	Husband	Asian-Pac-Islander	Male	0	0	
13	23	Private	122 272	Bachelors	13	Never-married	Adm-clerical	Own-child	White	Female	0	0	
14	32	Private	205 019	Assoc-acdm	12	Never-married	Sales	Not-in-family	Black	Male	0	0	
15	40	Private	121 772	Assoc-voc	11	Married-civ-spouse	Craft-repair	Husband	Asian-Pac-Islander	Male	0	0	
16	34	Private	245 487	7th-8th	4	Married-civ-spouse	Transport-moving	Husband	Amer-Indian-Eskimo	Male	0	0	
17	25	Self-emp-not-inc	176 756	HS-grad	9	Never-married	Farming-fishing	Own-child	White	Male	0	0	
18	32	Private	186 824	HS-grad	9	Never-married	Machine-op-inspct	Unmarried	White	Male	0	0	
19	38	Private	28 887	11th	7	Married-civ-spouse	Sales	Husband	White	Male	0	0	
20	43	Self-emp-not-inc	292 175	Masters	14	Divorced	Exec-managerial	Unmarried	White	Female	0	0	
21	40	Private	193 524	Doctorate	16	Married-civ-spouse	Prof-specialty	Husband	White	Male	0	0	
22	54	Private	302 146	HS-grad	9	Separated	Other-service	Unmarried	Black	Female	0	0	
23	35	Federal-gov	76 845	9th	5	Married-civ-spouse	Farming-fishing	Husband	Black	Male	0	0	
24	43	Private	117 037	11th	7	Married-civ-spouse	Transport-moving	Husband	White	Male	0	2 042	
25	59	Private	109 015	HS-grad	9	Divorced	Tech-support	Unmarried	White	Female	0	0	

48 842

Таблица Форма

Рисунок 2 — Загрузка данных в Logiном

Перейдем к обработке данных.

1.1.2 Предобработка и первичный анализ данных

В качестве предобработки необходимо избавиться от дубликатов,

обработать пропуски, закодировать категориальные данные, привести целевой признак в бинарный вид. Код для предобработки представлен в Листинге 2.

Листинг 2 — Предобработка данных в Python

```
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder,
StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

df = df.drop_duplicates()
for col in ['age', 'fnlwgt', 'education-num', 'capital-
gain', 'capital-loss', 'hours-per-week']:
    if df[col].isna().sum() > 0:
        median_value = df[col].median()
        df[col] = df[col].fillna(median_value)

for col in ['workclass', 'education', 'marital-
status', 'occupation', 'relationship', 'race', 'sex', 'native-
country']:
    if df[col].isna().sum() > 0:
        mode_value = df[col].mode()[0] if not
df[col].mode().empty else 'Unknown'
        df[col] = df[col].fillna(mode_value)

cat_cols = ['workclass', 'education', 'marital-
status', 'occupation', 'relationship', 'race', 'sex', 'native-
country']
num_cols = ['age', 'fnlwgt', 'education-num', 'capital-
gain', 'capital-loss', 'hours-per-week']

num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])
cat_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='constant',
fill_value='Unknown')),
    ('onehot', OneHotEncoder(handle_unknown='ignore',
sparse_output=False))
])
preprocessor = ColumnTransformer([
    ('num', num_pipeline, num_cols),
    ('cat', cat_pipeline, cat_cols)
])
X = df.drop(columns=['income'])
y = (df['income'].str.strip() == '>50K').astype(int)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, stratify=y, random_state=42)
print('Train:', X_train.shape, 'Test:', X_test.shape)
```

Предобработка данных в Loginom представлена на Рисунке 3.

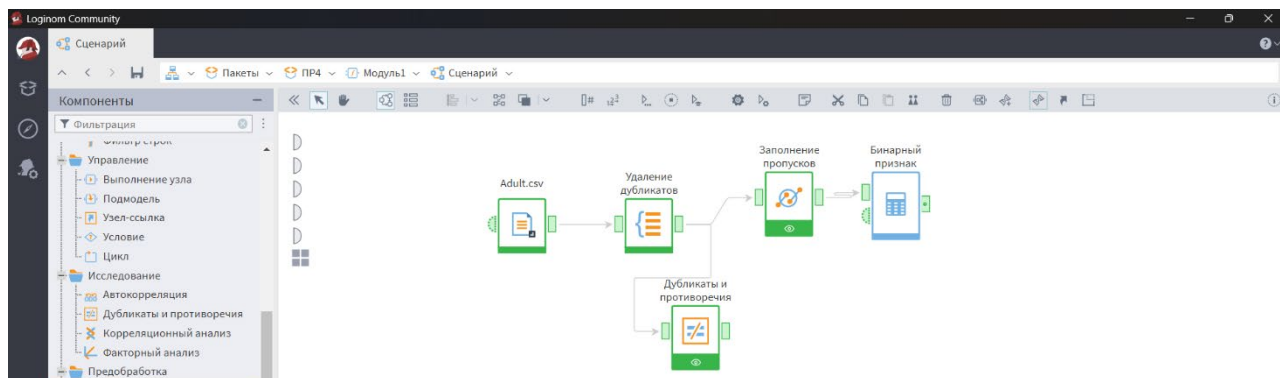


Рисунок 3 — Предобработка данных в Loginom

Дубликаты после обработки представлены на Рисунке 4.

Рисунок 4 — Отсутствие дубликатов

Целевой признак после обработки представлен на Рисунке 5.

Выходной набор данных												
#	оп	12 education-num	ab marital-status	ab occupation	ab relationship	ab race	ab sex	12 capital-gain	12 capital-loss	12 hours-per-week	ab native-country	0/1 Целевой признак
1		13	Never-married	Adm-clerical	Not-in-family	White	Male	2 174	0	40	United-States	false
2		13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	13	United-States	false
3		9	Divorced	Handlers-cleaners	Not-in-family	White	Male	0	0	40	United-States	false
4		7	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	0	0	40	United-States	false
5		13	Married-civ-spouse	Prof-specialty	Wife	Black	Female	0	0	40	Cuba	false
6		14	Married-civ-spouse	Exec-managerial	Wife	White	Female	0	0	40	United-States	false
7		5	Married-spouse-absent	Other-service	Not-in-family	Black	Female	0	0	16	Jamaica	false
8		9	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	45	United-States	true
9		14	Never-married	Prof-specialty	Not-in-family	White	Female	14 084	0	50	United-States	true
10		13	Married-civ-spouse	Exec-managerial	Husband	White	Male	5 178	0	40	United-States	true

Рисунок 5 — Бинарный целевой признак

Пропуски после обработки представлены на Рисунке 6.

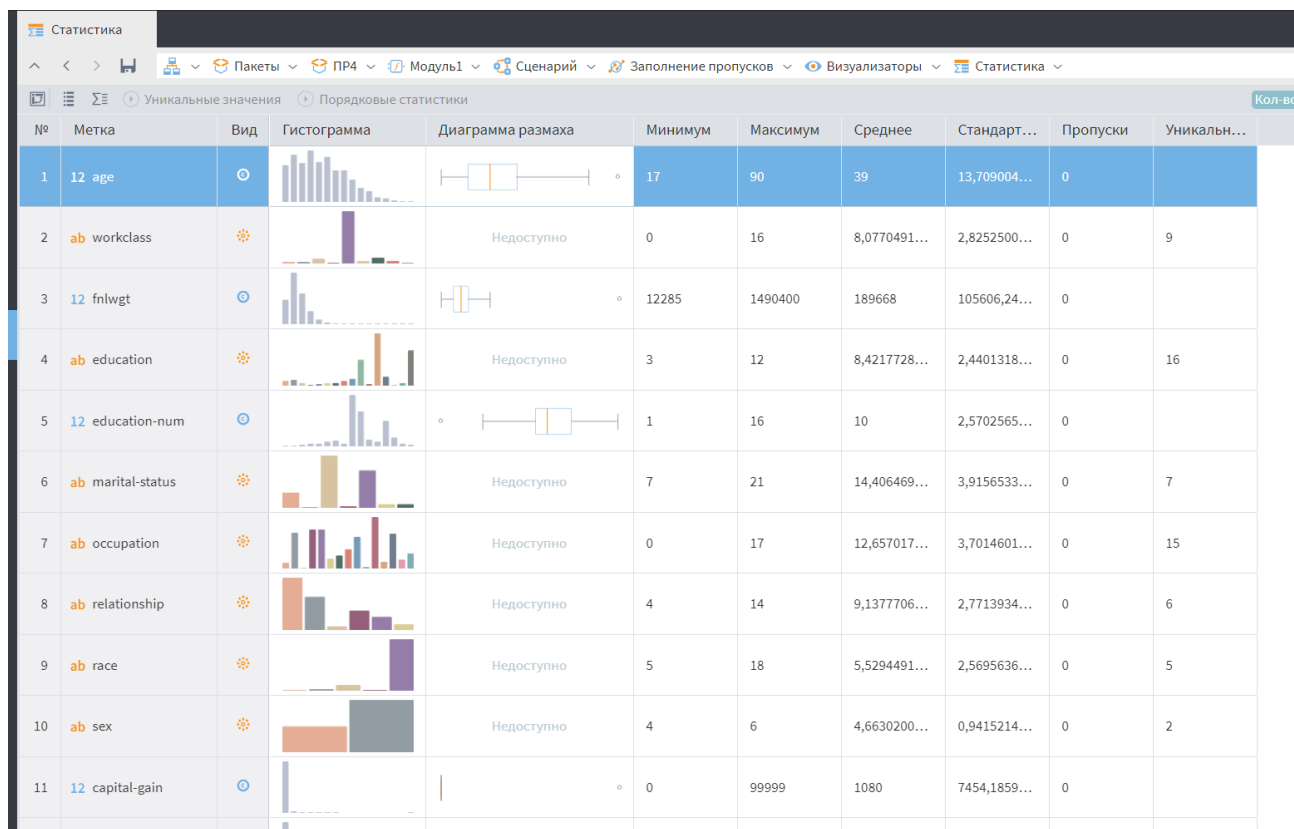


Рисунок 6 — Обработка пропусков

Перейдем к моделям логистической регрессии.

1.1.3 Построение модели логистической регрессии

Построенные в Logitom модели представлены на Рисунке 7.

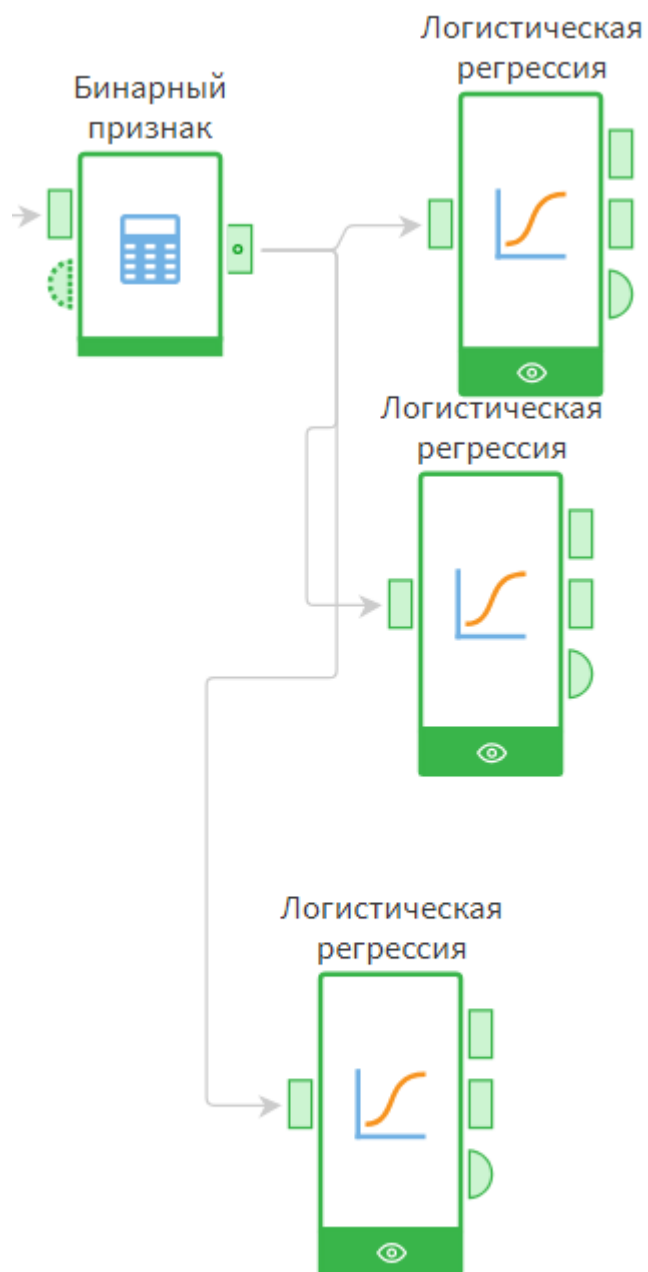


Рисунок 7 — Модели логистической регрессии Loginom

Код для моделей представлен в Листинге 3.

Листинг 3 — Модели логистической регрессии Python

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV,
StratifiedKFold
from sklearn.metrics import classification_report,
confusion_matrix, roc_auc_score
from sklearn.pipeline import Pipeline
```


Продолжение Листинга 3

```
cv = StratifiedKFold(n_splits=5, shuffle=True,
random_state=42)
pipe = Pipeline([('preproc', preprocessor), ('clf',
LogisticRegression(max_iter=5000))])

param_grid = [
    {'clf__penalty': ['l2'], 'clf__solver': ['lbfgs'], 'clf__C':
[0.01,0.1,1,10]},
    {'clf__penalty': ['l1'], 'clf__solver': ['liblinear'],
'clf__C': [0.01,0.1,1,10]}
]

gs = GridSearchCV(pipe, param_grid, cv=cv, scoring='f1',
n_jobs=-1, verbose=2)
gs.fit(X_train, y_train)

print('Best params:', gs.best_params_)
print('Best CV f1:', gs.best_score_)

best = gs.best_estimator_
y_pred = best.predict(X_test)
y_proba = best.predict_proba(X_test)[:,1]
print(classification_report(y_test, y_pred))
print('Confusion matrix:', confusion_matrix(y_test, y_pred))
print('ROC AUC:', roc_auc_score(y_test, y_proba))
```

Перейдем к сравнению оценок моделей.

1.1.4 Оценка моделей

Оценки классификации (для лучшей модели) в Logitom представлены на Рисунке 8.

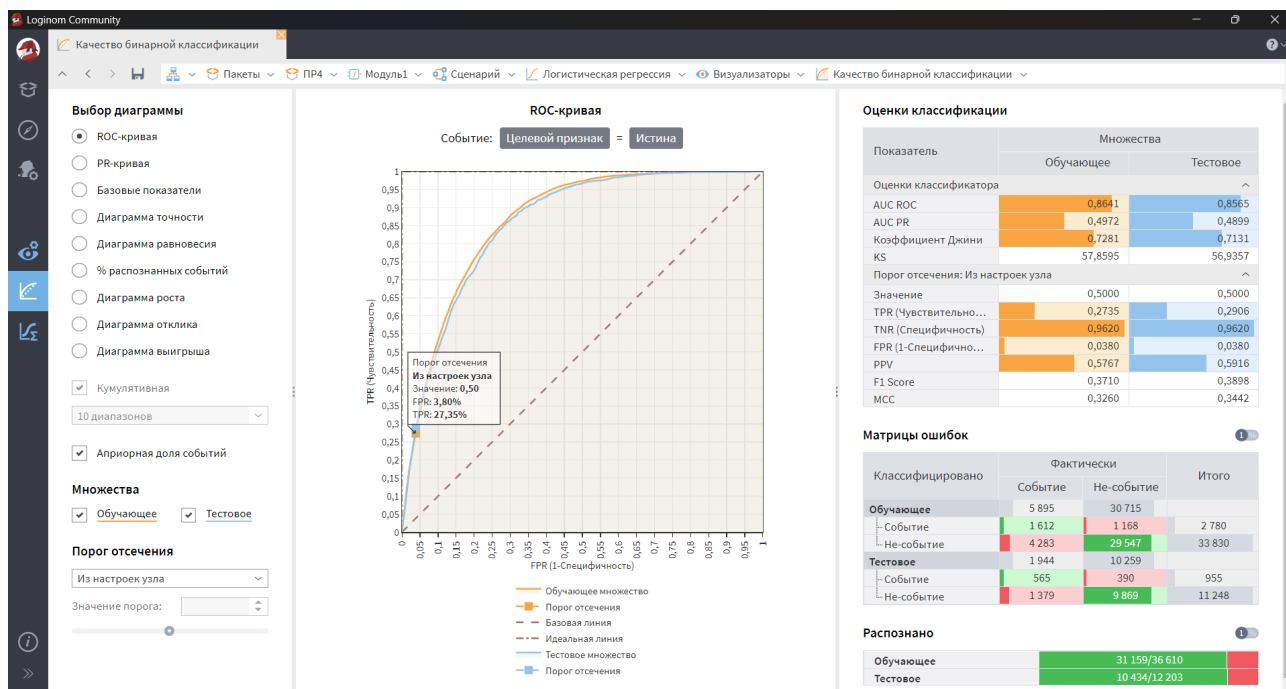


Рисунок 8 — Оценки классификации Loginom

При сравнительно большом ROC-AUC (0.8565 на тестовой выборке), метрика F1 довольно низка — 0.3898. Общая точность классификации около 0.84.

Результаты модели в Python представлены на Рисунке 9.

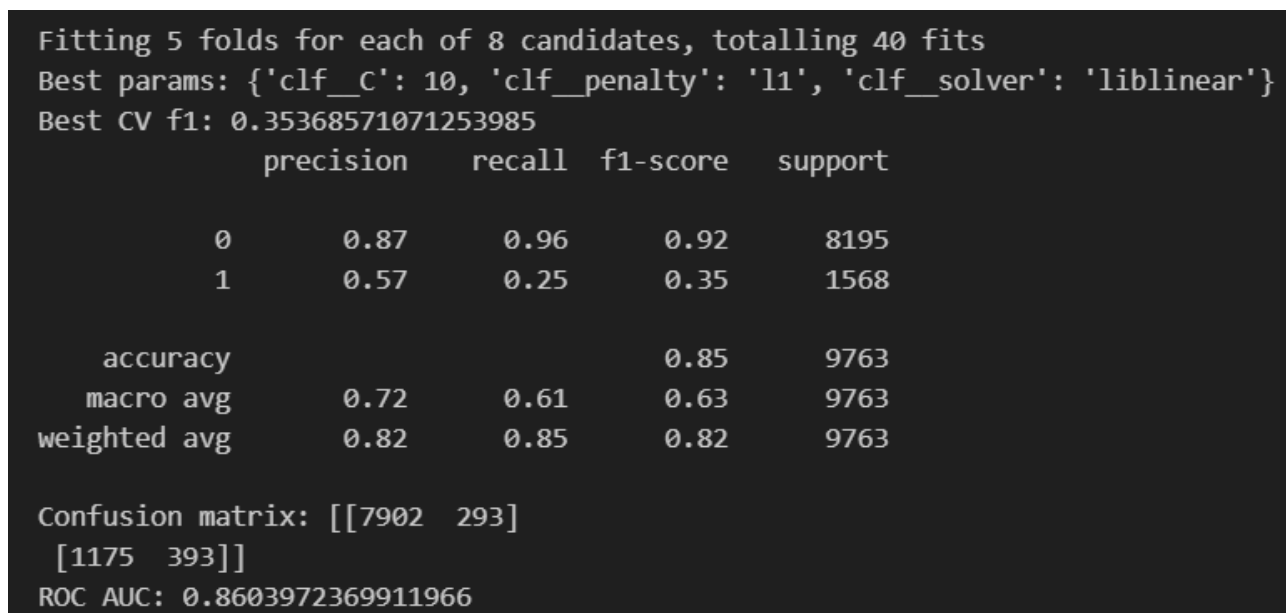


Рисунок 9 — Метрики модели в Python

Все метрики совсем немного выше, чем в Loginom, но общая картина идентичная.

1.1.5 Сравнение результатов и сред

Таким образом, простая модель логистической регрессии в Python выиграла лишь за счёт подобранных гиперпараметров, показав на 1% точности больше.

При этом в Logiном не нужно было для такого же результата писать код, а узлы были настроены гораздо быстрее, чем работа в Python.

Python удобен возможностью детальной настройки под конкретные задачи.

Logiном — универсальное средство для быстрых анализов при использовании простых моделей.

1.2 Задание 2. Классификация при несбалансированности классов

1.2.1 Выбор датасета

Выбранный датасет: «creditcard.csv» с пропусками. В нем присутствует задача классификации мошеннических операций по транзакциям, а следовательно большой дисбаланс классов.

Обзор данных в Python представлен на Рисунке 10.

```
cc = pd.read_csv('data/creditcard.csv')
cc.head()
```

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22	V23	V24	V25	V26	V27	V28
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928	0.128539	-0.189115	0.133558	-0.021053
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846	0.167170	0.125895	-0.008983	0.014724
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281	-0.327642	-0.139097	-0.055353	-0.059752
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575	0.647376	-0.221929	0.062723	0.061458
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267	-0.206010	0.502292	0.219422	0.215153

5 rows × 31 columns

Рисунок 10 — Обзор данных в Python

Загрузка данных в Logiном представлена на Рисунке 11.

Набор данных																								
#	V8	V9	V10	V11	V12	V13	V14	V15	V16	V17	V18	V19	V20	V21	V22	V23	V24	V25	V26	V27	V28	Amount	Class	
1	0,10	0,36	0,09	-0,55	-0,62	-0,99	-0,31	1,47	-0,47	0,21	0,03	0,40	0,25	-0,02	0,28	-0,11	0,07	0,13	-0,19	0,13	-0,02	149,62	0	
2	0,09	-0,26	-0,17	1,61	1,07	0,49	-0,14	0,64	0,46	-0,11	-0,18	-0,15	-0,07	-0,23	-0,64	0,10	-0,34	0,17	0,13	-0,01	0,01	2,69	0	
3	0,25	-1,51	0,21	0,62	0,07	0,72	-0,17	2,35	-2,89	1,11	-0,12	-2,26	0,52	0,25	0,77	0,91	-0,69	-0,33	-0,14	-0,06	-0,06	378,66	0	
4	0,38	-1,39	-0,05	-0,23	0,18	0,51	-0,29	-0,63	-1,06	-0,68	1,97	-1,23	-0,21	-0,11	0,01	-0,19	-1,18	0,65	-0,22	0,06	0,06	123,50	0	
5	-0,27	0,82	0,75	-0,82	0,54	1,35	-1,12	0,18	-0,45	-0,24	-0,04	0,80	0,41	-0,01	0,80	-0,14	0,14	-0,21	0,50	0,22	0,22	69,99	0	
6	0,26	-0,57	-0,37	1,34	0,36	-0,36	-0,14	0,52	0,40	-0,06	0,07	-0,03	0,08	-0,21	-0,56	-0,03	-0,37	-0,23	0,11	0,25	0,08	3,67	0	
7	0,08	0,46	-0,10	-1,42	-0,15	-0,75	0,17	0,05	-0,44	0,00	-0,61	-0,05	-0,22	-0,17	-0,27	-0,15	-0,78	0,75	-0,26	0,03	0,01	4,99	0	
8	-3,81	0,62	1,25	-0,62	0,29	1,76	-1,32	0,69	-0,08	-1,22	-0,36	0,32	-0,16	1,94	-1,02	0,06	-0,65	-0,42	-0,05	-1,21	-1,09	40,80	0	
9	0,85	-0,39	-0,41	-0,71	-0,11	-0,29	0,07	-0,33	-0,21	-0,50	0,12	0,57	0,05	-0,07	-0,27	-0,20	1,01	0,37	-0,38	0,01	0,14	93,20	0	
10	0,07	-0,74	-0,37	1,02	0,84	1,01	-0,44	0,15	0,74	-0,54	0,48	0,45	0,20	-0,25	-0,63	-0,12	-0,39	-0,07	0,09	0,25	0,08	3,68	0	

Рисунок 11 — Загрузка данных

Перейдем к распределению классов.

1.2.2 Проверка распределения классов

Проверка распределения классов в Python представлена на Рисунке 12.

```
print(cc.shape)
print(cc['Class'].value_counts())
print('Доля мошенничеств:', cc['Class'].mean())
```

✓ 0.0s

```
(284807, 31)
Class
0    284315
1      492
Name: count, dtype: int64
Доля мошенничеств: 0.001727485630620034
```

Рисунок 12 — Распределение классов Python

Узлы для проверки распределения классов в Logiном представлены на Рисунке 13.

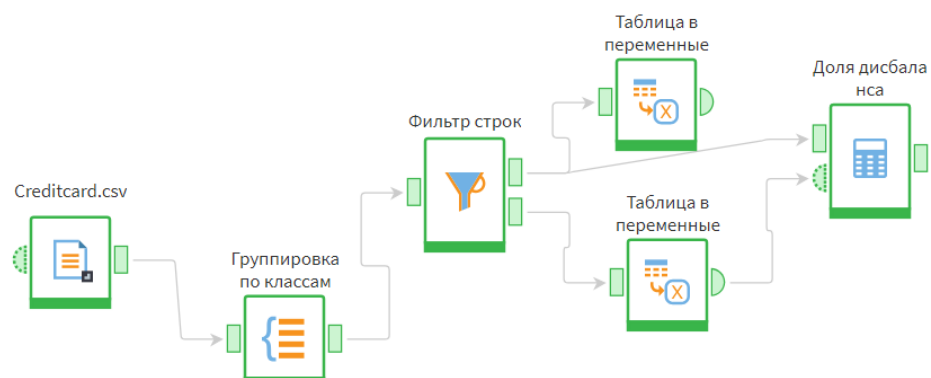


Рисунок 13 — Узлы проверки дисбаланса

Таким образом, количество мошеннических операций сильно ниже обычных.

1.2.3 Логистическая регрессия без балансировки классов

Код для логистической регрессии без учета дисбаланса классов представлен в Листинге 4.

Листинг 4 — Логистическая регрессия Python

```
X = cc.drop(columns=['Class'])
y = cc['Class']
X_train, X_test, y_train, y_test = train_test_split(X, y,
stratify=y, test_size=0.2, random_state=42)

num_cols = X.columns.tolist()
preproc = Pipeline([('imp',
SimpleImputer(strategy='median')), ('sc', StandardScaler())])

pipe = Pipeline([('preproc', preproc), ('clf',
LogisticRegression(max_iter=5000, class_weight=None))])
pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)
y_proba = pipe.predict_proba(X_test)[:,1]
print(classification_report(y_test, y_pred))
print('AUC:', roc_auc_score(y_test, y_proba))
```

Логистическая регрессия в Logitom представлена на Рисунке 14.

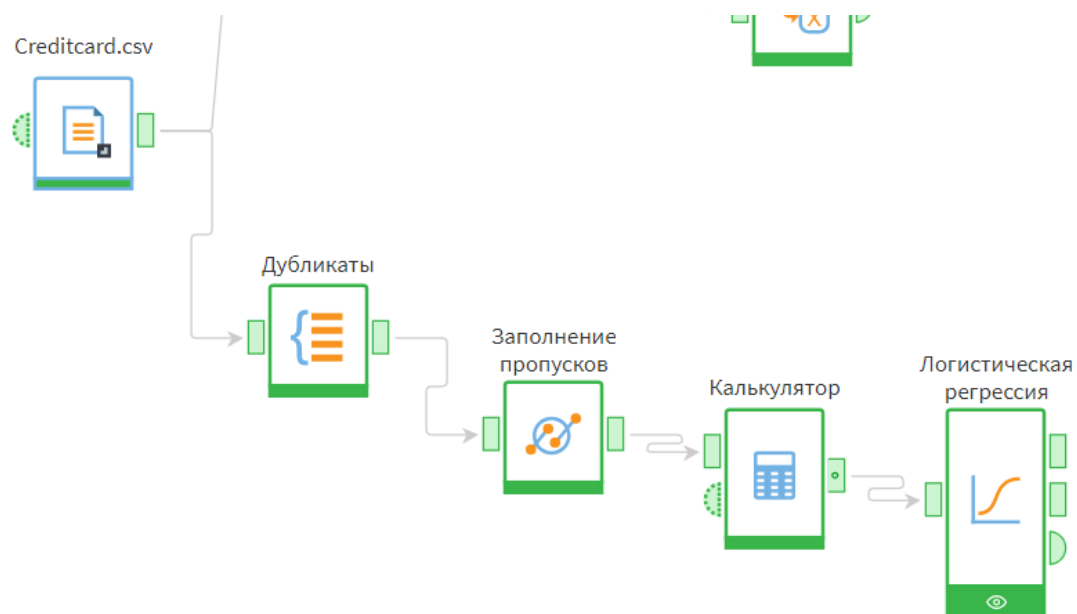


Рисунок 14 — Логистическая регрессия Loginom

Результат модели в Python представлен на Рисунке 15.

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.83	0.63	0.72	98
accuracy			1.00	56962
macro avg	0.91	0.82	0.86	56962
weighted avg	1.00	1.00	1.00	56962
AUC: 0.9605494455801453				

Рисунок 15 — Метрики модели в Python

Результат модели в Loginom представлен на Рисунке 16.

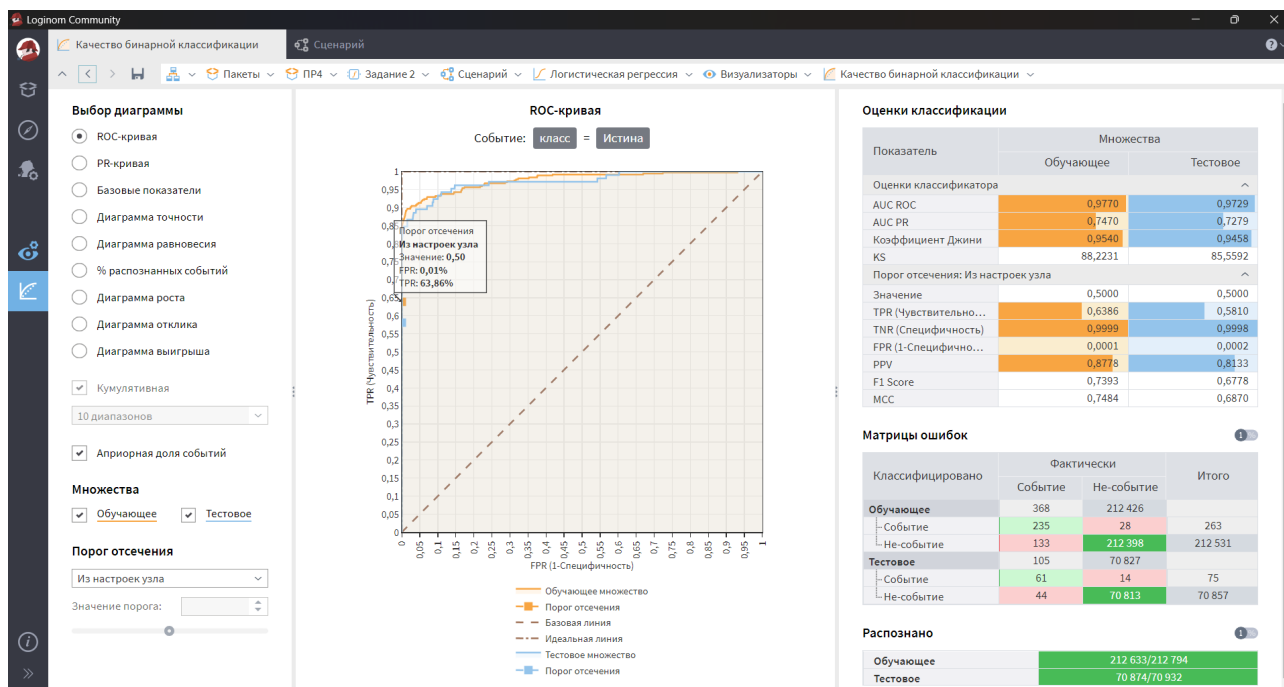


Рисунок 16 — Метрики модели Loginom

Перейдем к устранению дисбаланса.

1.2.4 Выбор метода устранения дисбаланса

Самым простым способом устранения будет поставить сбалансированный вес для классов. Модель и без этого справляется отлично, поэтому нет смысла перегружать её различными ансамблевыми или генеративными методами.

1.2.5 Повторное обучение

Код для обучения моделей со взвешенными классами представлен в Листинге 5.

Листинг 5 — Взвешенные классы

```

cw_pipe = Pipeline([('preproc', preproc), ('clf',
LogisticRegression(class_weight='balanced', max_iter=5000))])
cw_pipe.fit(X_train, y_train)
y_pred = cw_pipe.predict(X_test)

```

Продолжение Листинга 5

```
y_proba = cw_pipe.predict_proba(X_test)[: , 1]
print(classification_report(y_test, y_pred))
print('class_weight      AUC:',      roc_auc_score(y_test,
cw_pipe.predict_proba(X_test)[: , 1]))
```

Сбалансированные классы в Logitom представлены на Рисунке 17.

Поправка на долю событий

☒ На основе обучающего множества
 ☐ На основе тестового множества

Множество	Доля событий, %	Доля не-событий, %
Генеральное	0,02	99,98

Рисунок 17 — Балансировка классов

Результат новой модели Logitom представлен на Рисунке 18.

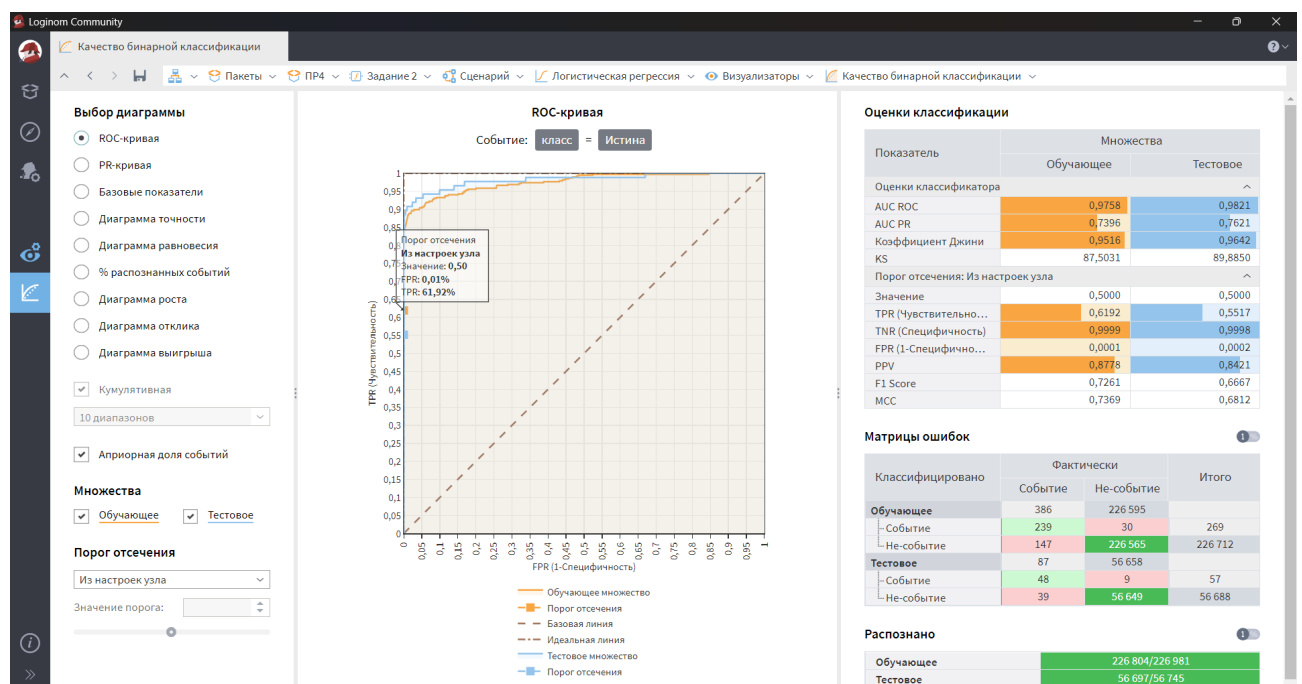


Рисунок 18 — Новая модель Logitom

Результат новой модели Python представлен на Рисунке 19.

	precision	recall	f1-score	support
0	1.00	0.98	0.99	56864
1	0.06	0.92	0.11	98
accuracy			0.98	56962
macro avg	0.53	0.95	0.55	56962
weighted avg	1.00	0.98	0.99	56962
class_weight AUC:	0.9720834996210077			

Рисунок 19 — Результат новой модели Python

Модель без балансировки классов (`class_weight=None`) демонстрирует высокую точность (accuracy ≈ 1.00), однако этот показатель вводит в заблуждение из-за сильного дисбаланса классов. На самом деле редко предсказывает положительный класс, чтобы избежать ложных срабатываний, в результате чего $\text{recall} = 0.63$. При этом $\text{precision} = 0.83$.

Применение `class_weight='balanced'` меняет поведение модели: она начинает активнее выявлять редкий класс, повышая recall до 0.92 (обнаружено 90 из 98 случаев мошенничества), но ценой резкого падения precision до 0.06 — теперь подавляющее большинство предсказаний положительного класса ложные. Accuracy снижается до 0.98, что ожидаемо и не является недостатком в условиях дисбаланса.

Одновременно с этим ROC-AUC возрастает с 0.9605 до 0.9721, что свидетельствует об улучшении способности модели ранжировать объекты по вероятности принадлежности к положительному классу.

Таким образом, использование сбалансированных весов позволяет значительно повысить чувствительность модели к редкому классу, что критически важно в задачах обнаружения мошенничества, несмотря на рост числа ложных срабатываний.

1.3 Задание 3. Многоклассовая логистическая регрессия

1.3.1 Выбор датасета

Выбранный датасет: «students.csv» с пропусками. В нем присутствует задача многоклассовой классификации: прогноз успеваемости учащихся в системе среднего образования (старшей школе).

Обзор данных представлен на Рисунке 20.

```
st = pd.read_csv('data/students.csv')

st.head()
```

✓ 0.0s

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	famrel	freetime	goout	Dalc	Walc	health	absences	G1	G2	G3
0	GP	F	18	U	GT3	A	4.0	4	at_home	teacher	...	4	3.0	4	1	1	3	4	0	11	11
1	GP	F	17	U	GT3	T	1.0	1	at_home	other	...	5	3.0	3	1	1	3	2	9	11	11
2	GP	F	15	U	LE3	T	1.0	1	at_home	other	...	4	3.0	2	2	3	3	6	12	13	12
3	GP	F	15	U	GT3	T	4.0	2	health	services	...	3	2.0	2	1	1	5	0	14	14	14
4	GP	F	16	U	GT3	T	3.0	3	other	other	...	4	3.0	2	1	2	5	0	11	13	13

5 rows × 33 columns

Рисунок 20 — Обзор данных Python

Загрузка данных в Logiном представлена на Рисунке 21.

Набор данных																				
#	ab school	ab sex	12 age	ab address	ab famsize	ab Pstatus	ab Medu	12 Fedu	ab Mjob	ab Fjob	ab reason	ab guardian	12 traveltime	12 studytime	12 failures	01 schoolsup	01 famsup	01 pe		
1	GP	F	18	U	GT3	A	4.0	4	at_home	teacher	course	mother	2	2	0	true	false	fr		
2	GP	F	17	U	GT3	T	1.0	1	at_home	other	course	father	1	2	0	false	true	fr		
3	GP	F	15	U	LE3	T	1.0	1	at_home	other	other	mother	1	2	0	true	false	fr		
4	GP	F	15	U	GT3	T	4.0	2	health	services	home	mother	1	3	0	false	true	fr		
5	GP	F	16	U	GT3	T	3.0	3	other	other	home	father	1	2	0	false	true	fr		
6	GP	M	16	U	LE3	T	4.0	3	services	other	reputation	mother	1	2	0	false		fr		
7	GP	M	16	U	LE3	T	2.0	2	other	other	home	mother	1	2	0	false	false	fr		
8	GP	F	17	U	GT3	A	4.0	4	other	teacher	home	mother	2	2	0	true	true	fr		
9	GP	M	15	U	LE3	A	3.0	2	services	other	home	mother	1	2	0	false	true	fr		
10	GP	M	15	U	GT3	T	3.0	4	other	other	home	mother	1	2	0	false	true	fr		

Рисунок 21 — Загрузка данных в Logiном

Перейдем к постройке моделей.

1.3.2 Постройка моделей

Ранжирование целевого признака в Python представлено в Листинге 6.

Листинг 6 — Ранжирование целевого признака

```
def grade_category(g):
    if g <= 10:
        return 'low'
    elif g <= 15:
        return 'medium'
    else:
        return 'high'

st['G3_cat'] = st['G3'].apply(grade_category)
y = st['G3_cat']
X = st.drop(columns=['G3', 'G3_cat'])

X = pd.get_dummies(X, drop_first=True)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
```

Ранжирование целевого признака в Loginom представлено на Рисунке 22.

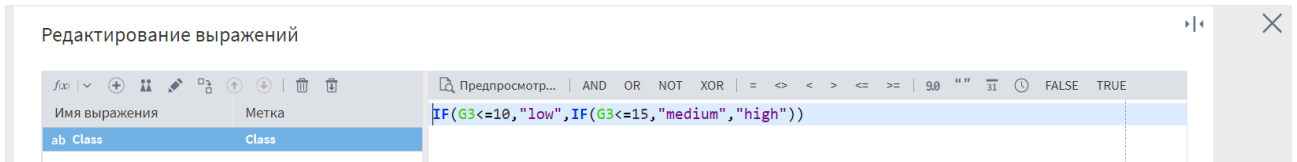


Рисунок 22 — Ранжирование целевого признака Loginom

Код для построения моделей представлен в Листинге 7.

Листинг 7 — Код для построения моделей в Python

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import classification_report,
accuracy_score, f1_score
import time

cv = StratifiedKFold(n_splits=5, shuffle=True,
random_state=42)

models = {
    'LogisticRegression': {
        'pipeline': Pipeline([('scaler', StandardScaler()),
('clf', LogisticRegression(max_iter=1000, random_state=42))]),
        'params': {
            'clf__C': [0.1, 1, 10],
            'clf__multi_class': ['ovr', 'multinomial'],
            'clf__solver': ['lbfgs', 'saga']
        }
    },
    'kNN': {
        'pipeline': Pipeline([('scaler', StandardScaler()),
('clf', KNeighborsClassifier())]),
        'params': {
            'clf__n_neighbors': [3, 5, 7, 9],
            'clf__weights': ['uniform', 'distance'],
            'clf__metric': ['euclidean', 'manhattan']
        }
    },
    'SVM': {
        'pipeline': Pipeline([('scaler', StandardScaler()),
('clf', SVC(random_state=42))]),
        'params': {
            'clf__C': [0.1, 1, 10],
            'clf__kernel': ['linear', 'rbf'],
            'clf__decision_function_shape': ['ovr']
        }
    }
}

results = []
```

Продолжение Листинга 7

```

for name, config in models.items():
    print(f"\nПодбор гиперпараметров для {name}...")

    start_time = time.time()
    grid = GridSearchCV(
        config['pipeline'],
        config['params'],
        cv=cv,
        scoring='f1_weighted',
        n_jobs=-1,
        verbose=0
    )
    grid.fit(X_train, y_train)
    fit_time = time.time() - start_time

    y_pred = grid.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred, average='weighted')

    results.append({
        'Model': name,
        'Best Params': grid.best_params_,
        'Accuracy': acc,
        'F1-weighted': f1,
        'Fit Time (s)': fit_time,
        'CV Best Score': grid.best_score_
    })

    print(f"{name} — F1: {f1:.4f}, Accuracy: {acc:.4f}, Время: {fit_time:.2f} c")

```

Метрики, полученные нейросетью в Loginom представлены на Рисунке 23.

Нейросеть (классификация) • Быстрый просмотр			
Выход нейросети		Сводка	
№	Имя	Метка	Значение
1	12 TotalSamples	Всего примеров	757
2	12 TotalSelectedSamples	Всего отобранных примеров	757
3	12 TrainSamples	Примеров в обучающем множестве	606
4	9.0 TrainRMSError	Среднеквадратическая ошибка на обучающем множестве	0,30
5	9.0 TrainClsErrorPercentage	Процент ошибок классификации на обучающем множестве	13,70
6	9.0 TrainAvgCE	Средняя перекрестная энтропия на обучающем множестве	1,67
7	12 TestSamples	Примеров в тестовом множестве	151
8	9.0 TestRMSError	Среднеквадратическая ошибка на тестовом множестве	0,36
9	9.0 TestClsErrorPercentage	Процент ошибок классификации на тестовом множестве	19,87
10	9.0 TestAvgCE	Средняя перекрестная энтропия на тестовом множестве	2,48
11	9.0 CVRMSError	Среднеквадратическая ошибка при кросс-валидации	0,34
12	9.0 CVClsErrorPercentage	Процент ошибок классификации при кросс-валидации	18,14
13	9.0 CVAvgCE	Средняя перекрестная энтропия при кросс-валидации	1,53

Рисунок 23 — Метрики в Loginom

График сравнения метрик моделей представлен на Рисунке 24.

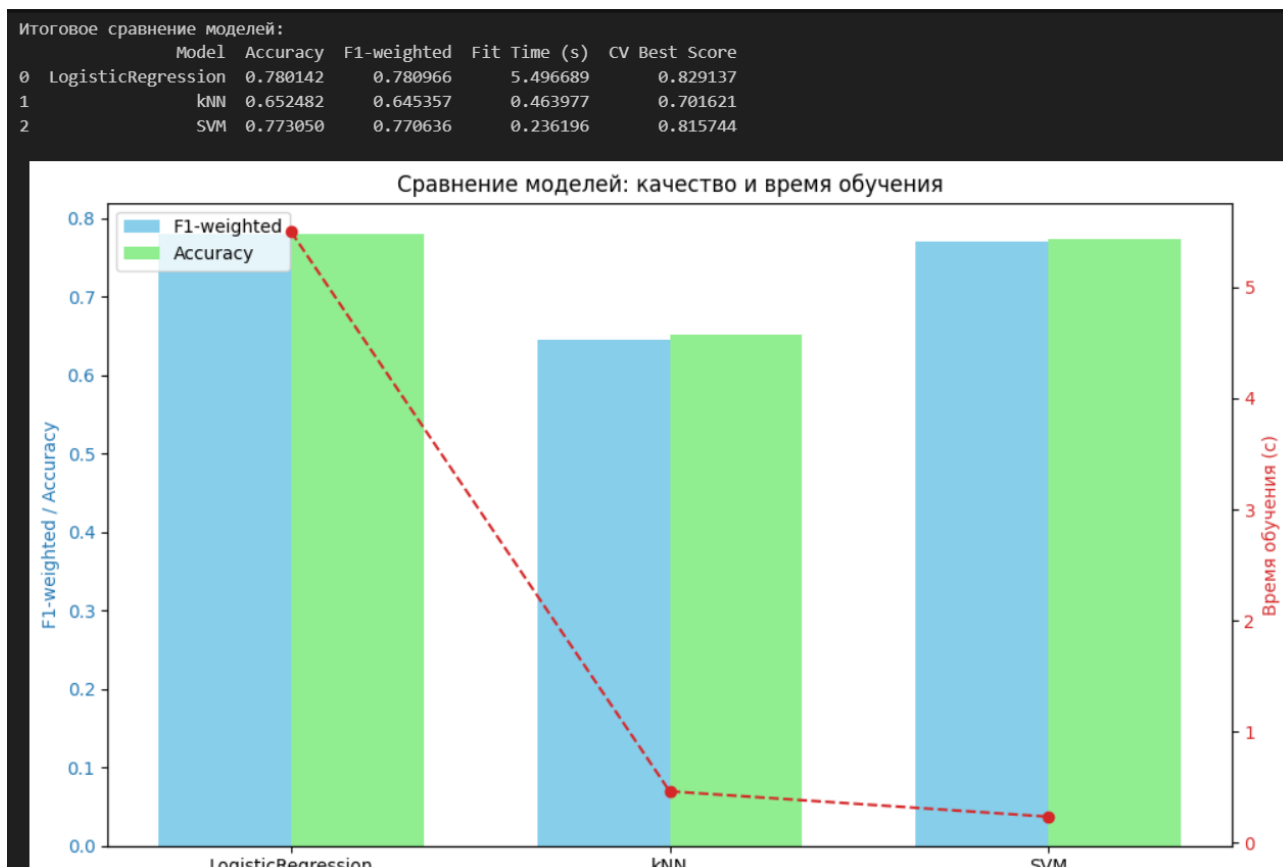


Рисунок 24 — Сравнение метрик моделей

Из трёх рассмотренных моделей наилучшее качество показала логистическая регрессия: она достигла самой высокой точности (78.0%) и F1-weighted (78.1%), а также продемонстрировала наилучший результат по кросс-валидации (82.9%). Модель SVM показала сопоставимые, но немного худшие метрики (77.3% accuracy, 77.1% F1), при этом обучалась значительно быстрее (0.24 с против 5.5 с у логистической регрессии). kNN оказался наименее эффективным как по качеству (65.2% accuracy, 64.5% F1), так и по обобщающей способности (CV Best Score — 70.2%), несмотря на высокую скорость обучения.

Таким образом, логистическая регрессия является предпочтительным выбором для данной задачи, если приоритетом является качество классификации, а не скорость обучения.

ВЫВОД

В ходе выполнения практической работы №4 по дисциплине «Технологии и инструментарий машинного обучения» были решены три ключевые задачи: построение линейных моделей на сбалансированном датасете, классификация при сильной несбалансированности классов и многоклассовая классификация с использованием различных алгоритмов.

В первом задании на датасете Adult была реализована логистическая регрессия с подбором гиперпараметров, что позволило достичь высокого ROC-AUC (≈ 0.86), однако низкого F1-мера из-за умеренного дисбаланса классов.

Во втором задании на датасете кредитных транзакций продемонстрировано, как использование сбалансированных весов (`class_weight='balanced'`) кардинально повышает recall для редкого класса (с 0.63 до 0.92), что критически важно в задачах обнаружения мошенничества, несмотря на снижение precision.

В третьем задании на студенческом датасете проведено сравнение трёх моделей — логистической регрессии, kNN и SVM — с подбором гиперпараметров через StratifiedKFold; лучшие результаты показала логистическая регрессия по точности (78.0%) и F1-weighted (78.1%), в то время как kNN оказался наименее эффективным. Также отмечено, что среда Logitom обеспечивает быстрое прототипирование и визуальную прозрачность, тогда как Python предоставляет гибкость и контроль над каждым этапом обработки и моделирования.

В совокупности работа подтвердила важность адекватной предобработки данных, учёта дисбаланса классов и обоснованного выбора метрик оценки в зависимости от предметной области.